



ENGINEERING FACULTY

## Final report

Course: Software Engineering - I

Student: Ruvims Ungeitis

St. code: 84951

Group: 4301BNA

Riga

2026

## Contents

|                                                      |          |
|------------------------------------------------------|----------|
| <b>Executive Summary .....</b>                       | <b>3</b> |
| <b>Engineering Harness: Constraining the AI.....</b> | <b>3</b> |
| <b>AGENTS.md Constraints .....</b>                   | <b>3</b> |
| <b>BDD Requirements .....</b>                        | <b>4</b> |
| <b>Architecture &amp; Logic .....</b>                | <b>5</b> |
| <b>Mermaid Architecture Diagram .....</b>            | <b>5</b> |
| <b>Mermaid Food Logging Flow .....</b>               | <b>5</b> |
| <b>Implemented Design Pattern .....</b>              | <b>6</b> |
| <b>UI &amp; Integration.....</b>                     | <b>7</b> |
| <b>Deployment Configuration .....</b>                | <b>8</b> |
| <b>PKM Insights &amp; Retrospective .....</b>        | <b>8</b> |
| <b>Conclusion.....</b>                               | <b>9</b> |

## Executive Summary

The Smart Nutrition & Fitness Tracker is a software engineering MVP designed to reduce the friction of food tracking. Traditional nutrition apps often require slow manual entry, complicated databases, and many screens before a user can understand what they ate. This project solves that problem with a lightweight application where a user can register, search for food, test a sample barcode flow, enter a natural-language meal description, add food to a daily log, and view calorie/macronutrient progress.

The target user is a student, beginner athlete, or health-conscious person who wants fast feedback rather than medical-grade diet planning. The system presents AI nutrition estimates as approximate, not as professional advice. The pitch from the first homework became the final product direction: make nutrition input faster, keep the interface simple, and use AI only where it improves speed.

Final outcome: a working full-stack MVP with a React Native/TypeScript frontend, FastAPI backend, external food/AI integrations, automated tests, Render deployment, and documented architectural decisions.

## Engineering Harness: Constraining the AI

The project treated AI as an engineering assistant, not as an unchecked code generator. The main harness was the project-level AGENTS.md file and the documentation-first rule. Before implementation decisions, the agent had to consult /docs, follow existing architecture, avoid unnecessary complexity, and explain trade-offs.

### AGENTS.md Constraints

Key Concepts:

- Focus on simplicity over complexity
- Prioritize user experience and speed of input
- AI output is approximate, not medically accurate
- System should be modular and scalable

Coding Rules:

- Use TypeScript for frontend
- Use clean architecture principles

- Keep functions small, reusable, single-responsibility
- Avoid unnecessary complexity, premature optimization, over-engineering

Documentation Rule:

- ALWAYS refer to the /docs folder before making decisions

### **BDD Requirements**

Feature: Food Logging

User story:

As a user, I want to enter the name of a food or scan its barcode, so that I can receive information about its nutritional values.

AC1 Search food by name

Given the user is on the food logging screen

When the user enters a food name and submits the request

Then the system shows nutritional information about that food

AC3 Food not found

Given the user is on the food logging screen

When the food cannot be found

Then the system shows a clear not-found message

These constraints reduced hallucinations because every generated change had to map back to documented scope: simple food tracking, approximate AI analysis, defensive error handling, and maintainable architecture.

## Architecture & Logic

The application follows a layered architecture. The frontend handles user interaction and state. The backend handles API routes, external service calls, and food-data normalization. PocketBase was used during development for authentication and persistence, while browser localStorage was used in the deployed web demo to avoid deploying a separate database service.

### Mermaid Architecture Diagram

flowchart TD

```
User[User] --> App[React Native Expo App]
App --> FoodModule[Food Logging Module]
FoodModule --> Reducer[Pure Reducer]
Reducer --> Command[Command Object]
Command --> Service[Frontend Food Service]
Service --> Backend[FastAPI Backend]
Backend --> FoodRoute[Food Route]
FoodRoute --> FoodService[Food Service]
FoodService --> Mapper[Pure Food Mapper]
FoodService --> ExternalAPI[Open Food Facts API]
FoodService --> OpenAI[OpenAI API]
```

### Mermaid Food Logging Flow

sequenceDiagram

actor User

participant App

participant Backend

participant FoodAPI as Open Food Facts

participant OpenAI as OpenAI API

User->>App: Enter food name or sample barcode

App->>Backend: Send food query

Backend->>FoodAPI: Request nutrition data

alt Food data found

Backend-->>App: Return calories and macros

```
App-->>User: Display nutrition card
else Food data not found
  Backend-->>App: Return not-found response
App-->>User: Show error message
end
User-->>App: Enter meal description
App-->>Backend: POST /food/analyze-text
Backend-->>OpenAI: Request structured estimate
```

### **Implemented Design Pattern**

The key pattern is the Command pattern in the food logging module. UI actions do not directly perform side effects. Instead, the reducer produces command objects such as `FETCH_FOOD_BY_NAME`, `FETCH_FOOD_BY_BARCODE`, `ANALYZE_MEAL_TEXT`, `SAVE_FOOD_TO_POCKETBASE`, `LOAD_SAVED_FOODS_FROM_POCKETBASE`, and `DELETE_FOOD_FROM_POCKETBASE`. The service layer executes those commands.

This design made the reducer easier to test because pure state transitions were separated from API calls. It also kept the feature scalable: adding persistence, deletion, weekly totals, or demo storage did not require mixing external operations directly into UI components.

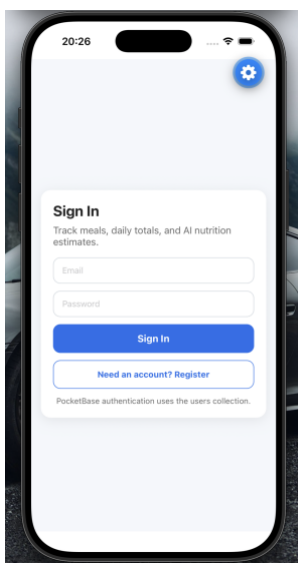
## UI & Integration

The UI was constrained by docs/design/design.md. The design rules required a simple vertical layout, consistent 16px spacing, blue primary actions, white cards, clear status messages, and no stacked primary buttons. The app follows these rules with authentication screens, a dashboard, a food logging card, nutrition result cards, logged-food rows, and explicit error/success states.

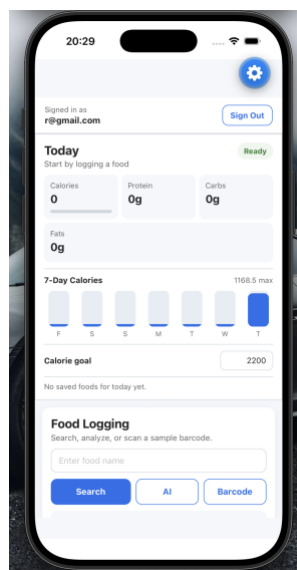
DESIGN.md rules:

- Primary: #1f6feb
- Background: #f4f7fb
- Card: #ffffff
- Body text: #222222
- Error: #ff4d4f
- Use vertical layout
- Keep spacing consistent
- Empty input -> show error
- Valid input -> show loading
- Success -> show food data

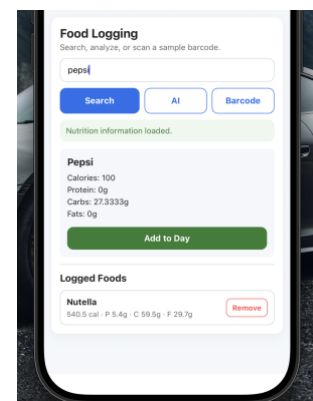
Integration work connected the React Native frontend to the FastAPI backend through a normalized API service. The frontend calls food search, barcode lookup, and AI analysis endpoints, while the backend calls Open Food Facts and OpenAI. Defensive mapping handles missing nutrition values so incomplete external data does not break the user flow.



Sign-in screen.



Dashboard and 7-day chart.



Food search and logged food.

## Deployment Configuration

The app is hosted on Render as two services: a static frontend site and a backend web service. The frontend is exported from Expo for web. The backend runs FastAPI through Uvicorn. The deployed frontend points to the deployed backend through EXPO\_PUBLIC\_API\_URL.

| Layer         | Configuration                                                        | Purpose                                          |
|---------------|----------------------------------------------------------------------|--------------------------------------------------|
| Backend       | backend/requirements.txt: fastapi,<br>uvicorn[standard], httpx       | Installs API framework, server, and HTTP client. |
| Backend start | python3 -m uvicorn app.main:app --host 0.0.0.0 -<br>-port \$PORT     | Runs FastAPI on Render's assigned port.          |
| Frontend      | npm install && npx expo export --platform web                        | Builds Expo React Native as a static web app.    |
| Environment   | EXPO_PUBLIC_API_URL,<br>OPENAI_API_KEY,<br>EXPO_PUBLIC_USE_DEMO_DATA | Connects services and supports demo persistence. |

PocketBase was used locally during development, but the final web demo uses browser localStorage for user accounts and nutrition records. This was an intentional MVP deployment decision: it kept the demo free and testable, while preserving the original PocketBase architecture for future production work.

## PKM Insights & Retrospective

The personal knowledge management experiment helped turn scattered implementation work into traceable engineering decisions. The project documented prompts, architecture notes, next-step plans, setup instructions, smoke tests, and feature requirements. This record made the final report easier because the report could be based on the actual development journey, not memory.

**Biggest AI challenge:** keeping AI suggestions inside the project scope instead of accepting unnecessary services, abstractions, or features.

**Overengineering:** PocketBase was useful locally, but deploying it separately increased complexity. localStorage was the pragmatic demo trade-off.

**Hallucinations:** AI output had to be checked against source files, tests, documentation, and manual behavior before being trusted.

**Future engineer role:** the engineer defines constraints, validates outputs, protects architecture, writes tests, and decides acceptable trade-offs.



## **Conclusion**

The project completed a full SDLC path: idea, requirements, architecture, implementation, UI integration, testing, deployment, and retrospective. The final system demonstrates a working AI-assisted nutrition tracking MVP and documents the trade-offs required to make it usable, understandable, and deployable within the project scope.